

Reinforcement Learning

從入門到放棄



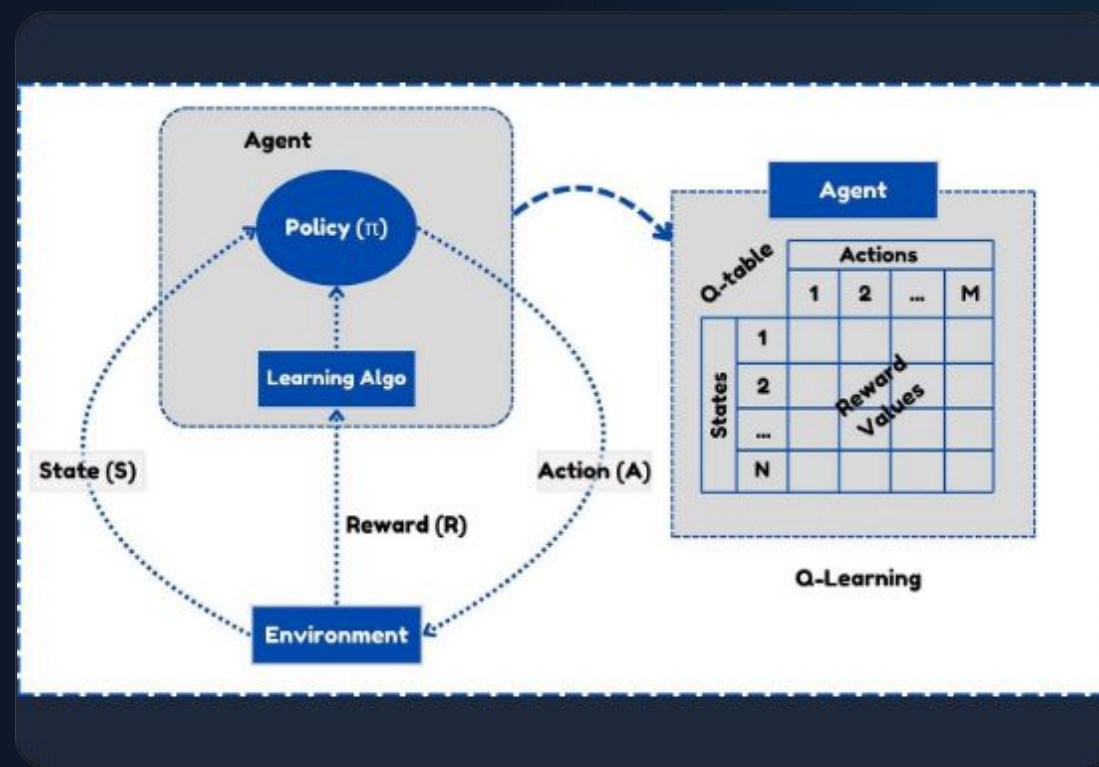
第一單元：基礎理論

理解強化學習的核心框架：馬可夫決策過程 (MDP) 與其背後的數學動力。

馬可夫決策過程 (MDP)

五元組定義 (S, A, P, R, γ)

- > S (States): 環境的所有可能狀態集合。
- > A (Actions): 智能體可以採取的動作集合。
- > P (Probability): 狀態轉移概率，描述環境動態。
- > R (Reward): 執行動作後獲得的即時獎勵。
- > γ (Discount): 折扣因子，權衡遠期利益。



貝爾曼方程 (BELLMAN EQUATION)

強化學習的靈魂：當前價值的遞迴定義。

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V^{\pi}(s')]]$$

期望報酬

當前價值等於即時獎勵加未來折扣價值。

最佳性

定義了尋找最佳策略的核心條件。



第二單元：演算法大觀園

探索 Value-based, Policy-based 到融合兩者的 Actor-Critic 架構。

基於價值的 RL (VALUE-BASED)

核心：學習評估狀態-動作對的「價值」。

- 目標：逼近最優 Q 函數 $Q^*(s, a)$ 。
- 決策：根據 Q 值選取最優動作。
- 例子：Q-Learning, DQN。

Q-Learning 更新

$$Q(s, a) \leftarrow Q + \alpha [r + \gamma \max_{a'} Q(s', a') - Q]$$

基於策略的 RL (POLICY-BASED)

不同於 Value-based 去估算 $Q(s, a)$ ，我們直接參數化策略：

$$\pi_{\theta} (a | s) = P [A_t = a | S_t = s ; \theta]$$

- **輸入：** 當前狀態 s 。
- **輸出：** 動作空間上的概率分佈。
- **權重 θ ：** 神經網路的參數，決定了智能體的「性格」。

為什麼要這麼做？

1. **連續動作空間：** 無法在無限動作中取 $\max$ 。
2. **隨機策略：** 能夠學習到最優的隨機行為（如：剪刀石頭布）。
3. **更優的收斂屬性：** 策略變化通常比價值函數變化更平滑。

策略梯度定理 (The Gradient Theorem)

我們希望最大化總預期獎勵 $J(\theta)$ ，因此沿著梯度方向更新：

$$\nabla_{\theta} J(\theta) \propto E_{s \sim d_{\pi}, a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q_{\pi}(s, a)]$$

$\log$ 技巧

將概率的乘積轉換為梯度的求和，方便神經網路反向傳播。

權重分配

如果一個動作的回報 Q 是正的，我們就增加該動作出現的概率。

挑戰：高方差 (High Variance)

Policy Gradient 最大的敵人是**不穩定**。

- 單次採樣的軌跡 (Trajectory) 隨機性很大。
- 一個極端好或極端壞的獎勵會讓策略劇烈震盪。
- **解決方案**：引入 Baseline。

引入優勢函數 (Advantage)

$$A(s, a) = Q(s, a) - V(s)$$

只學習「比平均水平好多少」，這正是 Actor-Critic 的基礎。

從 PG 到 PPO

為了克服傳統 Policy Gradient 的更新步長難以控制的問題：

TRPO (前身)

限制新舊策略之間的 KL 散度，確保「安全更新」。

PPO (當前主流)

使用 **Clipped Objective**。如果新策略偏離舊策略太遠，就強制「切斷」梯度。

$$L = \min (r_t A_t , \text{clip} (r_t , 1 - \epsilon , 1 + \epsilon) A_t)$$

這就是為什麼 Isaac Gym 能夠在極短時間內訓練出穩定動作的原因。

ACTOR-CRITIC

Actor (演員)

負責「行動」。學習策略
 $\pi(a|s)$ 。

Critic (評論家)

負責「打分」。評估 Actor 的表現。

優勢

結合了低方差與直接優化的特性。

ON-POLICY VS. OFF-POLICY

維度	On-policy (同策)	Off-policy (異策)
定義	學習與互動使用同一策略。	學習與互動策略可以不同。
數據利用	數據即產即用，無法回放。	可使用 Replay Buffer 回放數據。
例子	SARSA, PPO	Q-Learning, SAC

在線強化學習 (On-line RL)

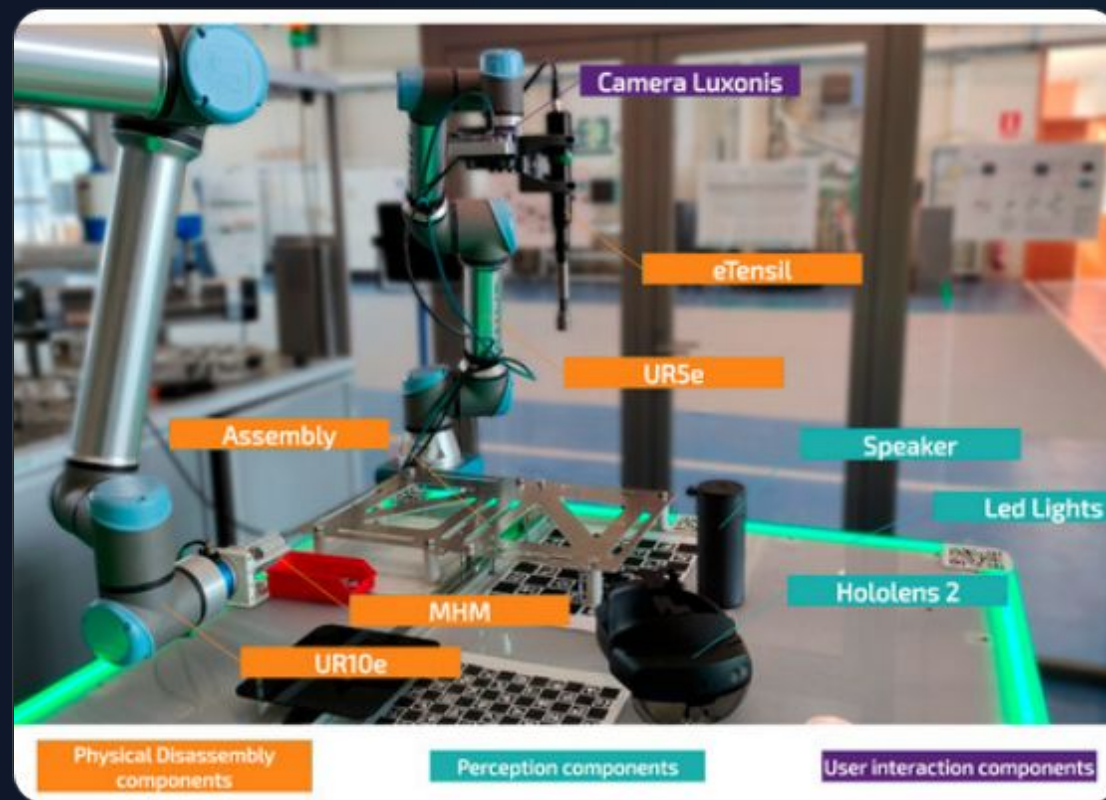
即時交互與試錯

這是最經典的模式：智能體在環境中邊走邊學。

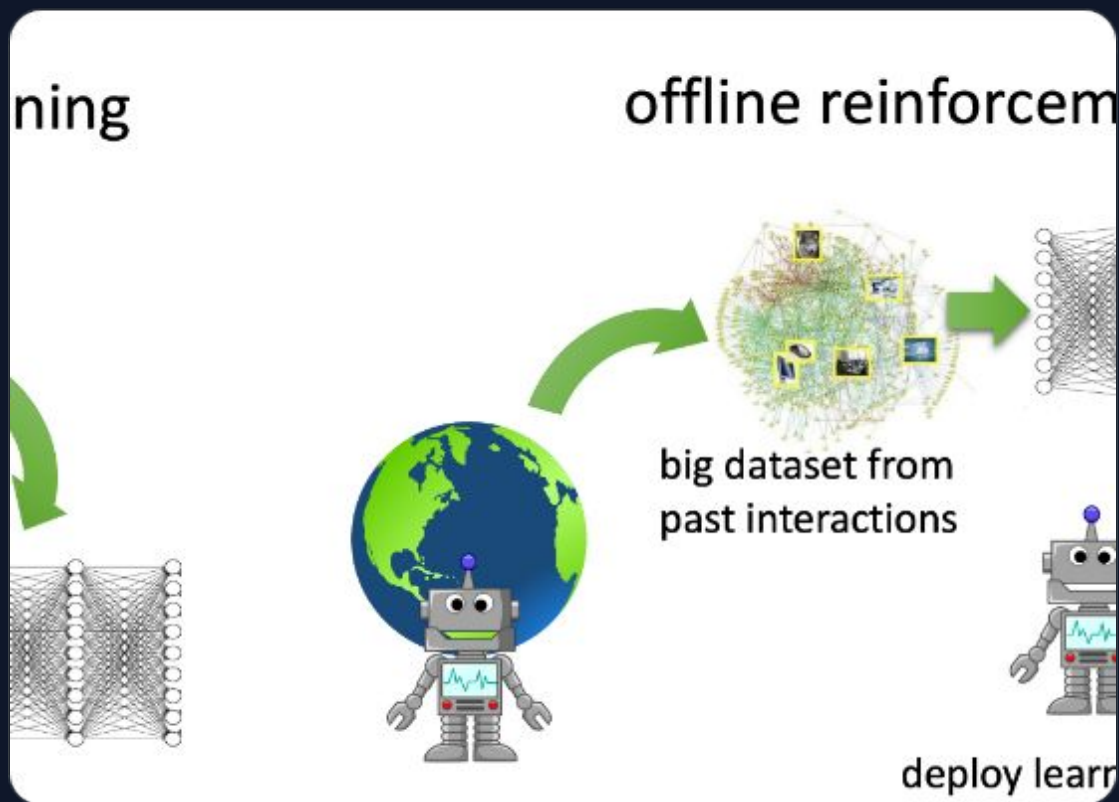
🔄 **實時採樣：**每一小步都在收集新數據。

🧪 **探索特性：**必須主動探索未知狀態。

⚠️ **缺點：**對環境依賴高，且具有安全風險。



離線強化學習 (Off-line RL)



從歷史數據中挖掘價值

智能體不再與環境交互，而是專注於「靜態數據集」。

 **Batch Learning:** 類似於監督式學習的流程。

 **零風險:** 無需在真實世界進行危險探索。

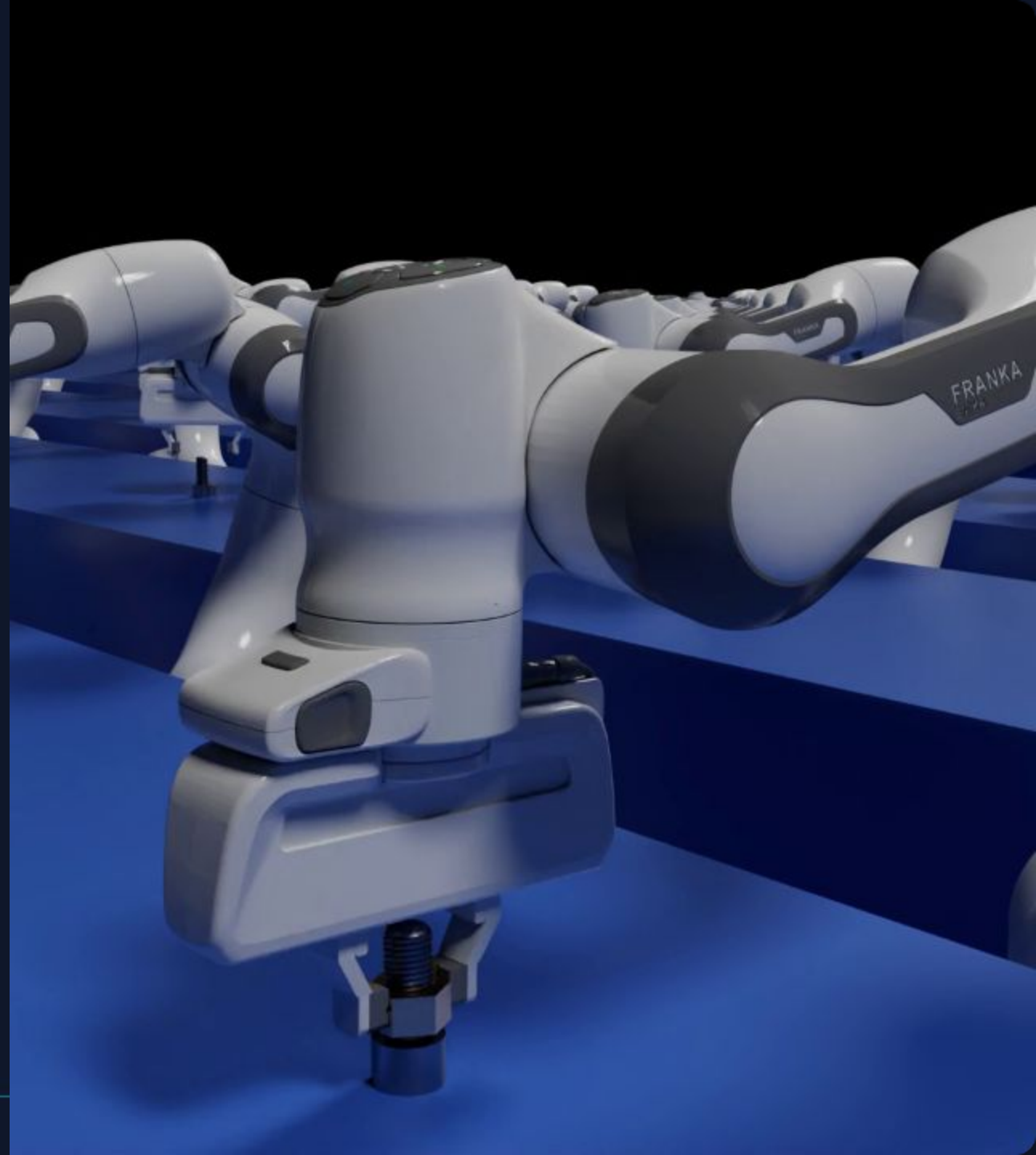
 **高效率:** 可重複利用海量過往數據。

ON-LINE VS. OFF-LINE

維度	On-line RL	Off-line (Batch) RL
數據來源	當前策略與環境交互產生的流數據	固定的歷史數據集 (Logs/Demo)
訓練成本	高 (需實時交互、依賴模擬器)	低 (僅需 GPU 與靜態文件)
安全性	具備不確定性 (可能損壞硬體)	絕對安全 (隔離訓練)
核心挑戰	樣本效率、穩定性	分佈偏移 (Distributional Shift)

為什麼 ISAAC LAB 選擇 PPO?

- **超大規模並行性：** PPO (On-policy) 能夠處理每秒數萬次的 GPU 並行採樣，且不需維護巨大的離線緩存池。
- **訓練穩定性：** 機器人模擬中容易出現極端狀態，PPO 的 Clipped Objective 確保了策略更新不會崩潰。
- **Sim-to-Real 強健性：** PPO 在解決如 ANYmal 四足機器人行走等複雜控制任務中展現了極佳的泛化能力。
- **GPU 原生優化：** PPO 的算法結構非常適合在 CUDA 上進行向量化運算，極大化了 NVIDIA 硬體性能。



實戰視角：如何選擇？



離散/簡單任務

優先考慮 **DQN** 或 **Q-Learning**，
簡單且樣本效率高。



機器人連續控制

PPO 是首選（穩定、易並行），
若要追求樣本效率則選 **SAC**。



大數據學習

若有無窮數據，**On-policy** 更穩定
；數據有限則必選 **Off-policy**。